

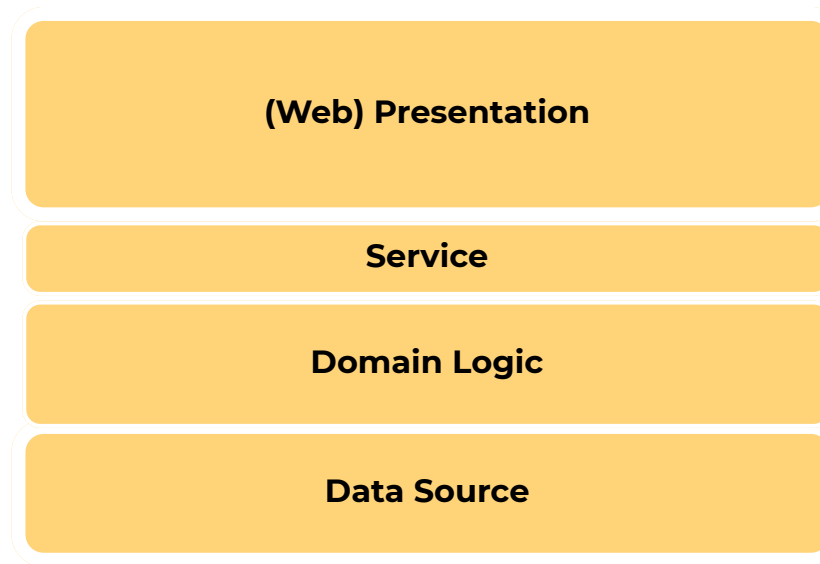
Camada de Apresentação

Rich Client Applications

Camada de Apresentação

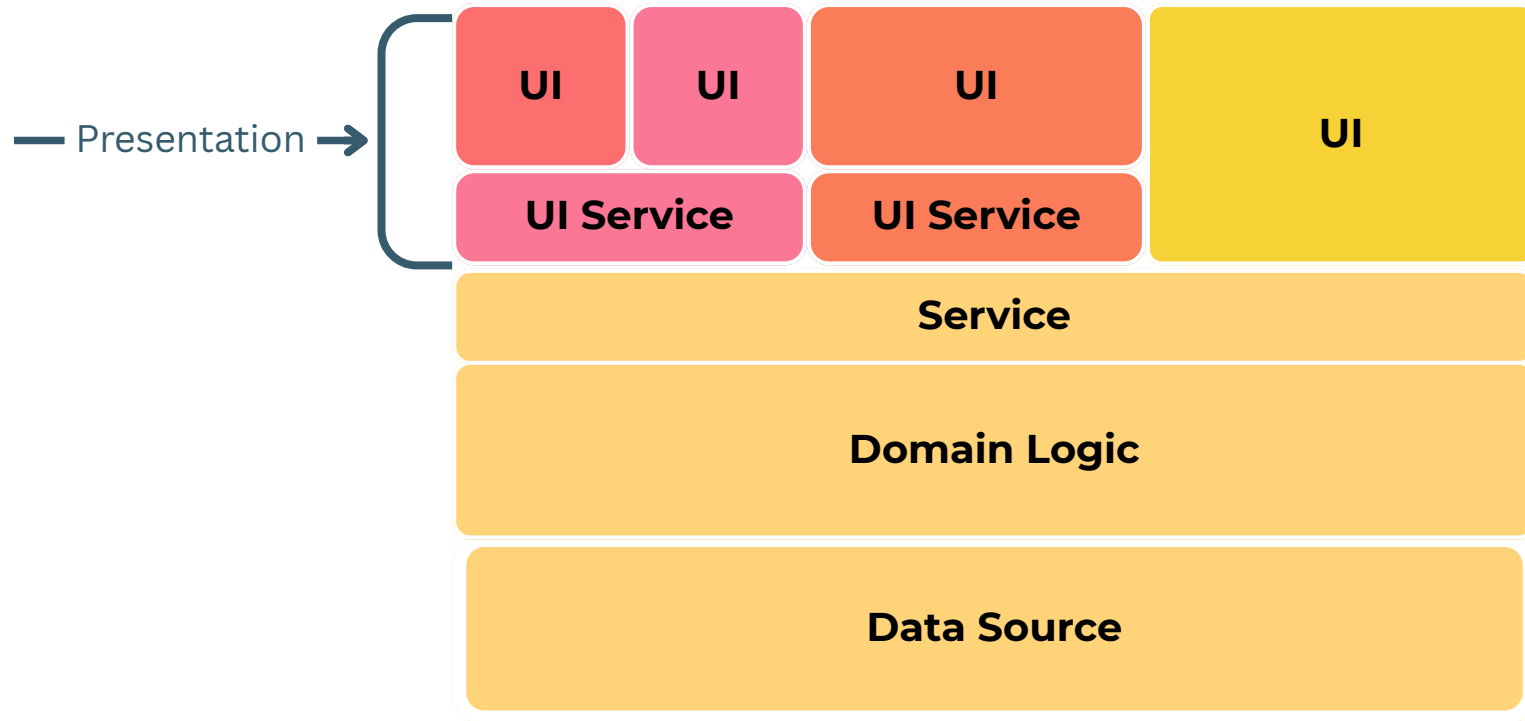
caso particular

- Até agora estivemos focados na construção da camada
 - de apresentação de aplicações web Java (o cliente é um browser)
 - em que todo o código específico da aplicação corre na **mesma máquina**



Camada de Apresentação

caso geral

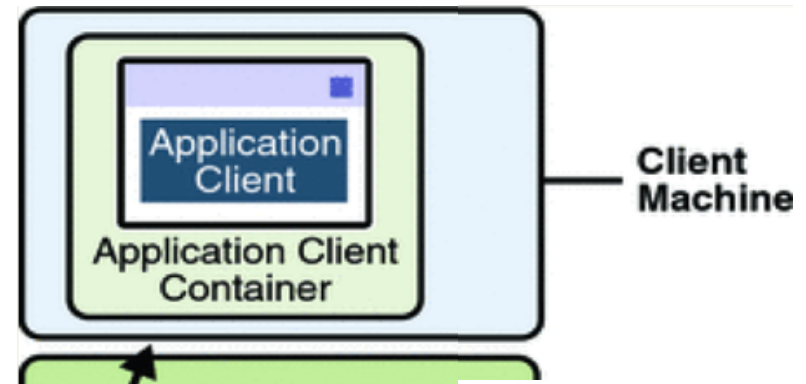
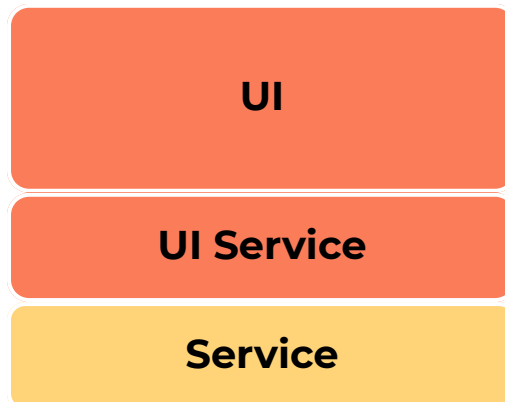


Rich Client Applications

- *Rich Client UIs*
 - permitem experiências de utilização de ricas
 - interativas
 - **grande desempenho**
- Normalmente, usa-se **aplicação cliente** (*Rich Client-Application*), ou seja o componente do UI que:
 - corre na máquina do utilizador
 - interage com um outro componente que presta os serviços da aplicação (e.g., numa arquitetura cliente-servidor)
- O código destes clientes faz parte da camada de apresentação da aplicação, a qual contém também o código que expõe os serviços que estão disponíveis para aquela interface (remotamente)

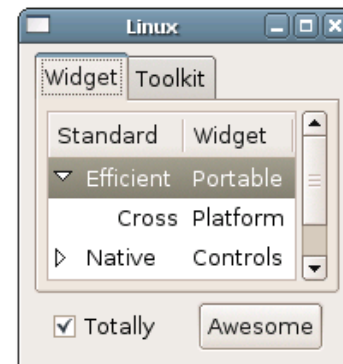
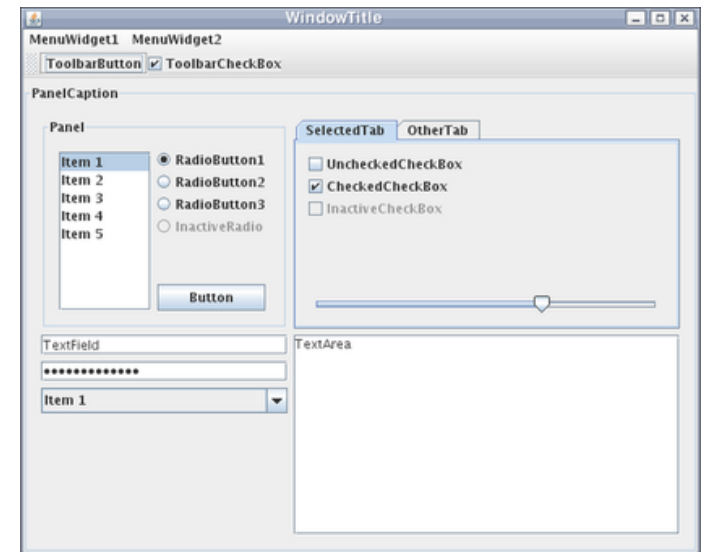
Rich Client Applications

- Nas aplicações Java EE estes UIs podem ser construídos como **Application Client Components** que:
 - executam dentro de um **Client Container** nas máquinas dos clientes
 - comunicam com os serviços do *backend* recorrendo a **RMI**
 - ganhando acesso aos serviços remotos através de **injeção de dependências**



GUI toolkits

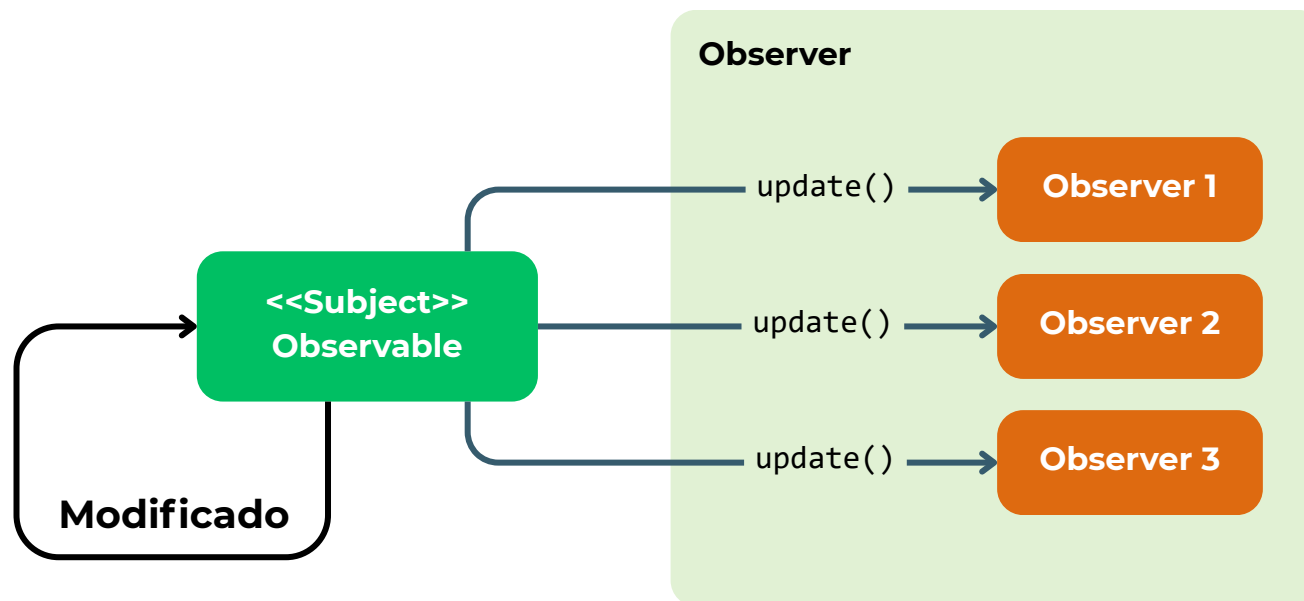
- Baseados em tecnologia Java
- Podem ser usados para programar os **Application Client Components**
- Entre os mais populares temos
 - **Swing**, Oracle JFC
 - **SWT** (*Standard Widget Toolkit*), Eclipse native widgets
 - **JavaFX**, Oracle



GUI toolkits

- **Padrões de Desenho**

- **View tree** é o principal padrão de estruturação para GUIs, usado para output, input e layout
- Recorrendo ao padrão **Observer** são usados **Eventos** e **Listeners** para comunicar ações de input e também para a comunicação entre a *view* e o *(view)model*



GUI toolkits

- **Padrões de Desenho**

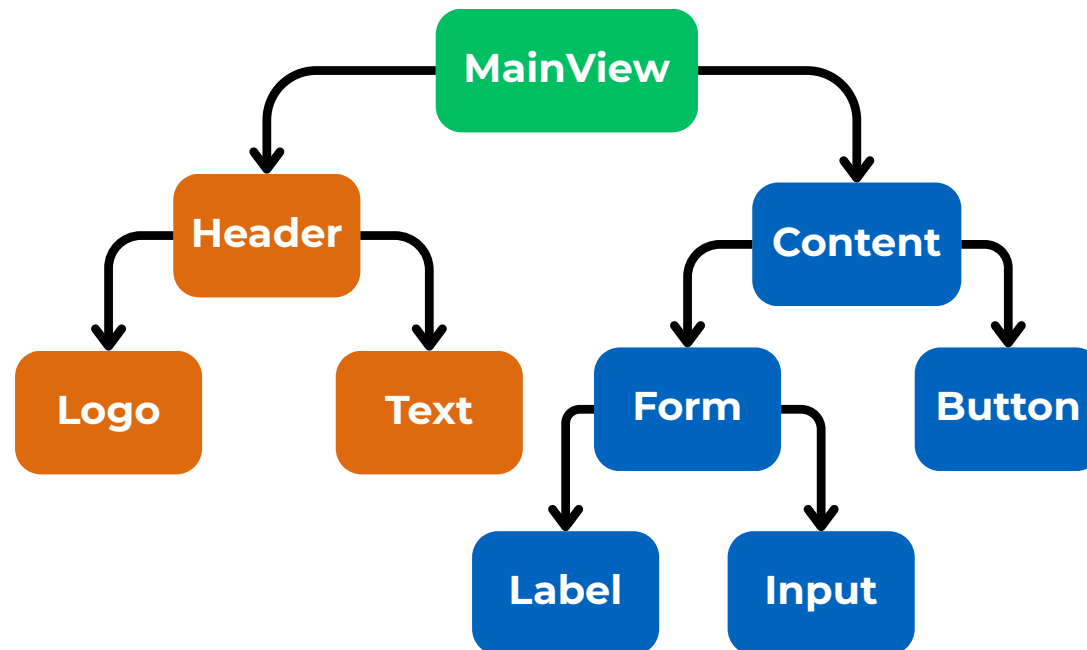
- Variante do **MVC** baseada em *widgets*/componentes-UI que gerem tanto o seu input como o seu output e que resultam da fusão da *view* e do *(view)controller* (que não são portanto independentes, mas antes altamente ligados um ao outro)
- O *backend* separado do GUI

- Principais **abordagens** à programação GUI

- Procedimental (how)
- Declarativa (what)
- Manipulação direta do interface (criar a *view tree*)

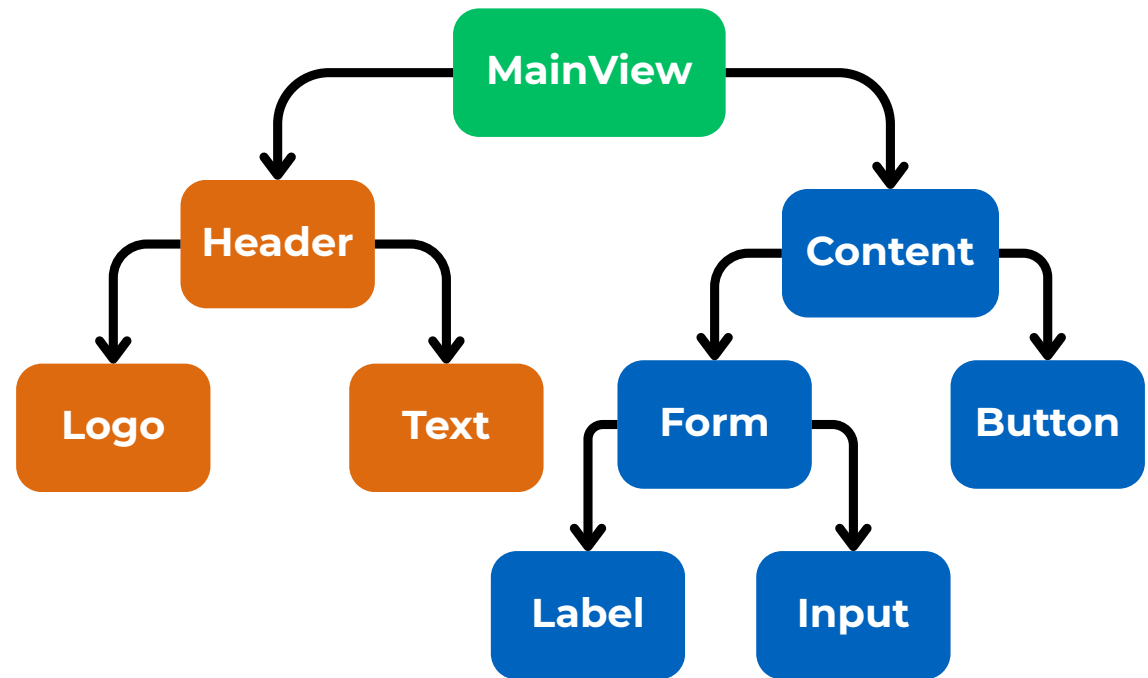
View Tree

- Uma view é um componente visual
 - JComponent no Swing
 - Node/Element no HTML
- Estrutura alinhada com estrutura conceptual dos dados
 - Se um nó da árvore não é mostrado, seus filhos também não são



View Tree

- Esta árvore é usada para:
 - **Output:** mudanças na *view tree* + algoritmos que automaticamente redesenham as views afectadas
 - **Input:** *event listeners/handlers* associados às views → processa os input do teclado/rato/ecrã
 - **Layout:** algoritmos que atravessam a árvore e conseguem automaticamente calcular os tamanhos das diferentes *views* e o seu posicionamento



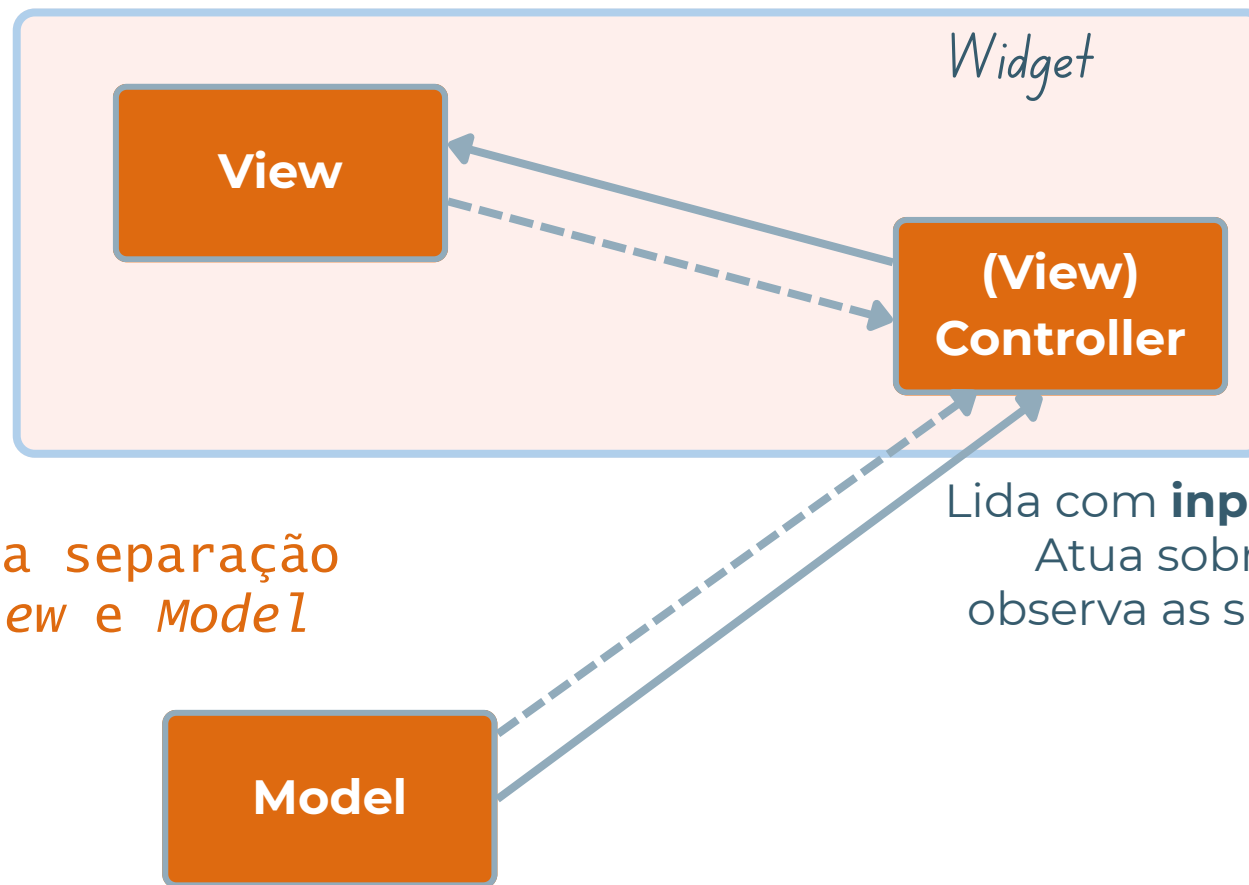
Input: Events + Listeners

- Várias fontes de **eventos**
 - o rato, o teclado, etc
- Um ou mais **listeners** registam interesse em eventos de uma dada fonte, fornecendo uma função para ser chamada quando o evento ocorre (*callback function* ou *event handler*)
 - Interesse pode ser limitado a eventos com determinadas características
 - Este interesse pode ser desativado/ativado dinamicamente
- Quando o evento ocorre, a fonte do evento distribui-o para todos os **listeners** interessados, invocando as respetivas funções

MVC baseado em widgets/componentes-UI

Lida com **output** e com **input** de baixo nível; passa a ser responsável pelo estado transiente

Mantida a separação entre *View* e *Model*



Lida com **input** de **alto nível**
Atua sobre o model e observa as suas mudanças

JavaFX

- Facilita a criação de UIs que se comportam de uma forma consistente em diferentes plataformas para *Rich Internet Applications*
- Substitui o **Swing** como o *toolkit* recomendado para as aplicações *desktop* Java com muitas vantagens
 - design mais consistente
 - mais *features*
 - conjunto rico de APIs de gráficos e media (audio, vídeo e animações)
- Utiliza diferentes *rendering engines* nativos, de acordo com a plataforma de execução (OpenGL, Direct3D)

JavaFX

- Permite o desenho de um interface gráfico usando
 - um ficheiro XML para definir o **visual layout** (FXML)
 - um CSS para definir a **aparência gráfica**
 - à semelhança do que acontece nas aplicações web
- IDEs oferecem ferramentas que permitem desenvolvimento rápido
 - Gluon Scene Builder
- A programação do UI é baseada
 - no tratamento de **eventos**
 - em ***bindings* de propriedades**

JavaFX

conceitos chave

- **Stage**

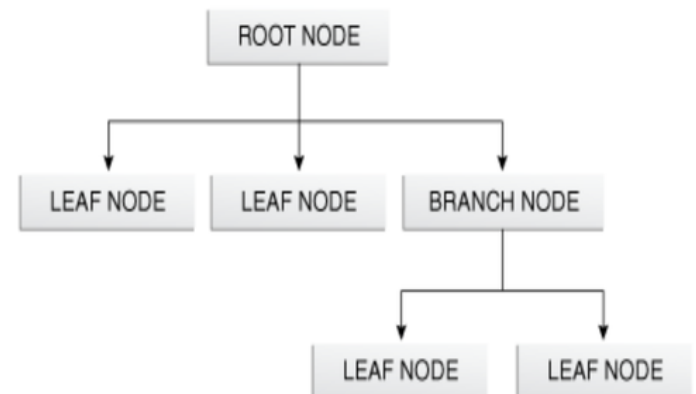
- onde é feita a apresentação da aplicação

- **Scene**

- contentor dos elementos que compõem uma composição que podemos colocar num *stage*

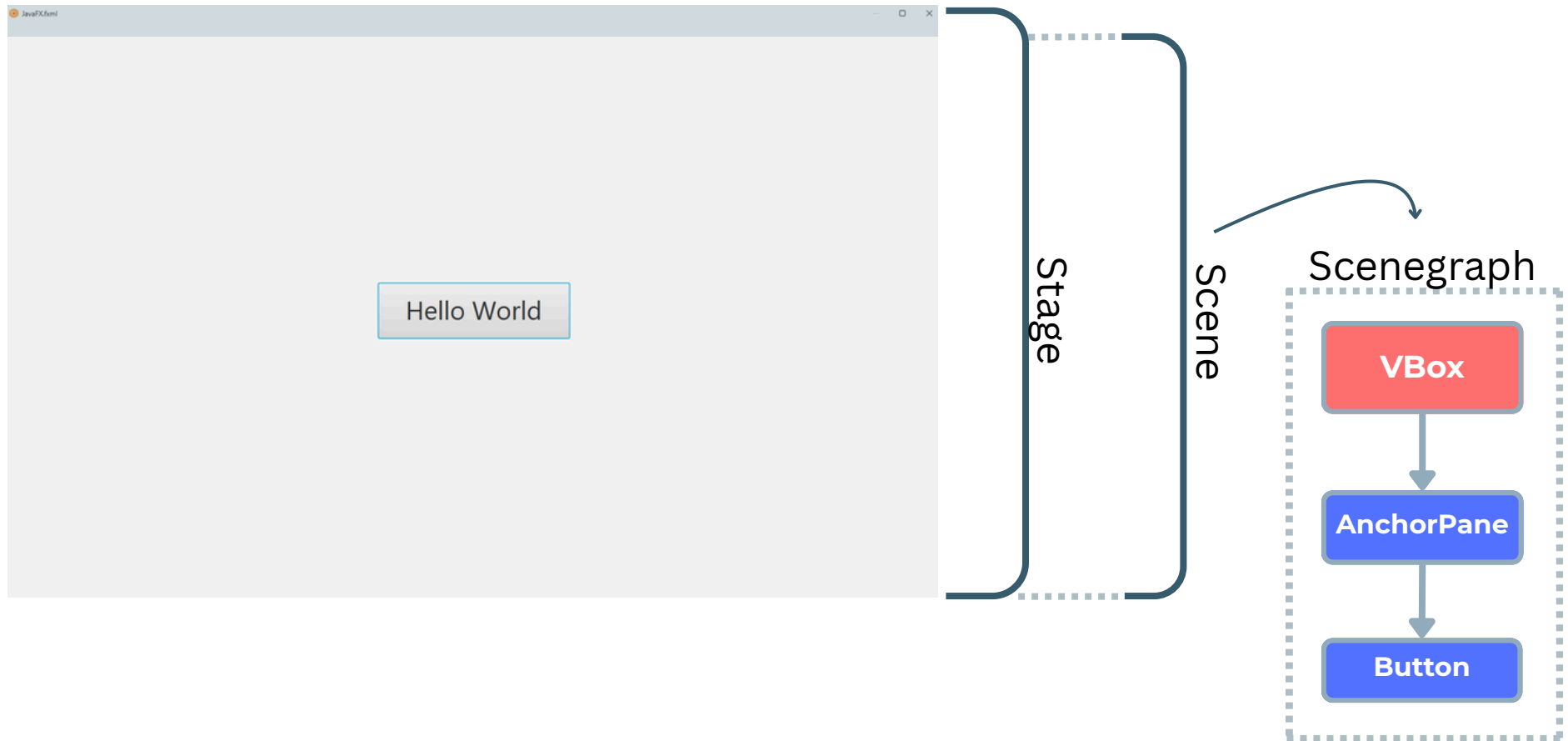
- **Node (of Scene Graph)**

- são os elementos de uma **Scene**, com um determinado *layout* e comportamento de interação
- podem ser contentores de outros nós



JavaFX

Anatomia da aplicação



JavaFX

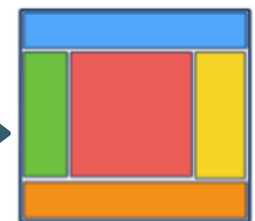
Anatomia da aplicação

- Uma aplicação JavaFX contém um ou mais **stages**
 - correspondem às janelas
 - o **primary stage** é construído automaticamente
- Cada **stage** tem associada uma **scene**
- Cada **scene** tem associado um **scene graph**
 - os nós são objetos do tipo **node**
 - os arcos representam a relação de *containment*

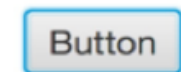
JavaFX

Anatomia da aplicação

- Uma aplicação JavaFX contém um ou mais **stages**
 - correspondem às janelas
 - o **primary stage** é construído automaticamente
- Cada **stage** tem associada uma **scene**
- Cada **scene** tem associado um **scene graph**
 - os nós são objetos do tipo **node**
 - os arcos representam a relação de *containment*
- Na hierarquia cuja raiz é **Node** temos tipos que representam
 - **Layout containers**: elementos que têm associado um algoritmo de apresentação dos elementos que contêm
 - **UI controls**: elementos com funcionalidades de controlo

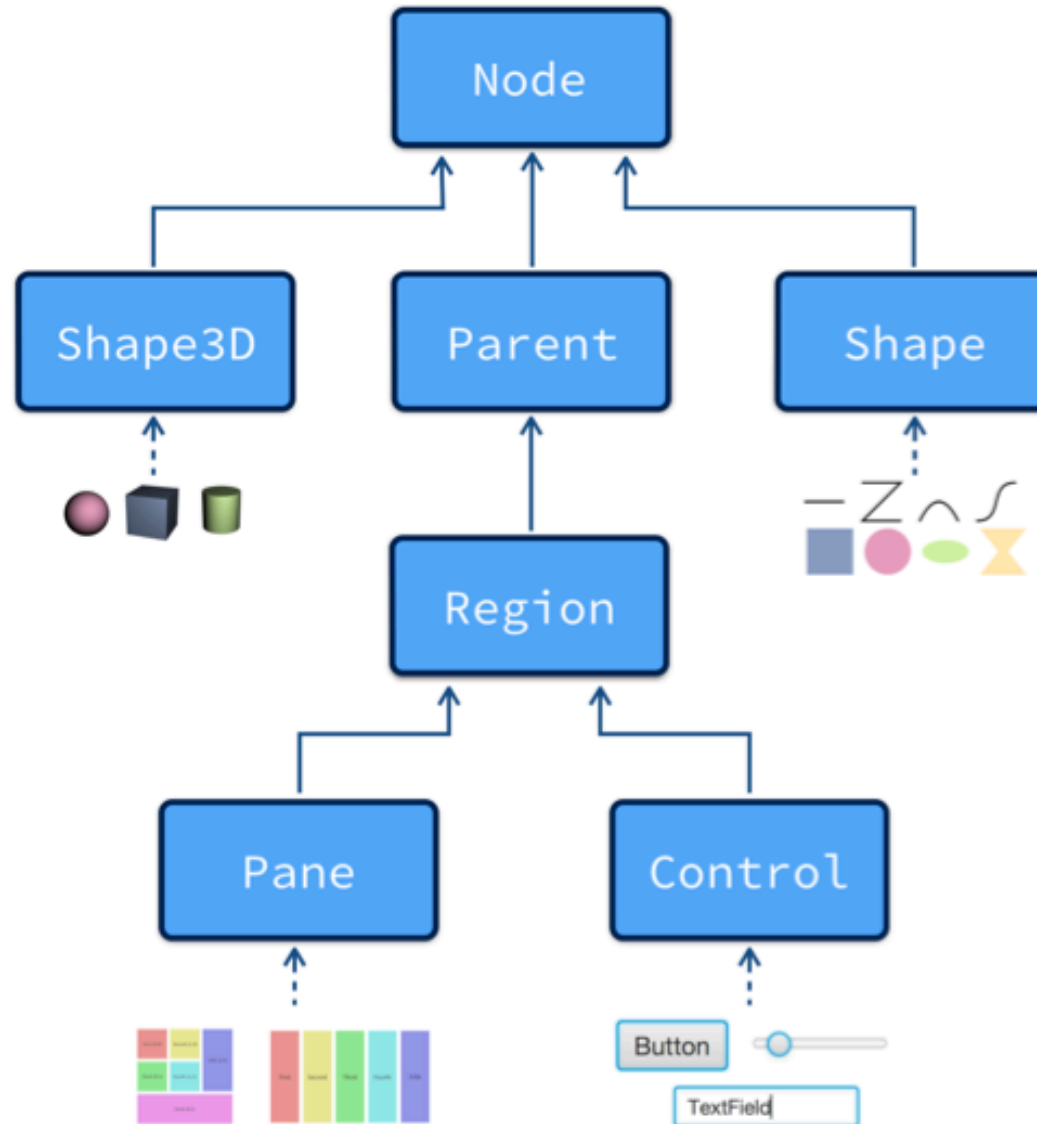


BorderPane



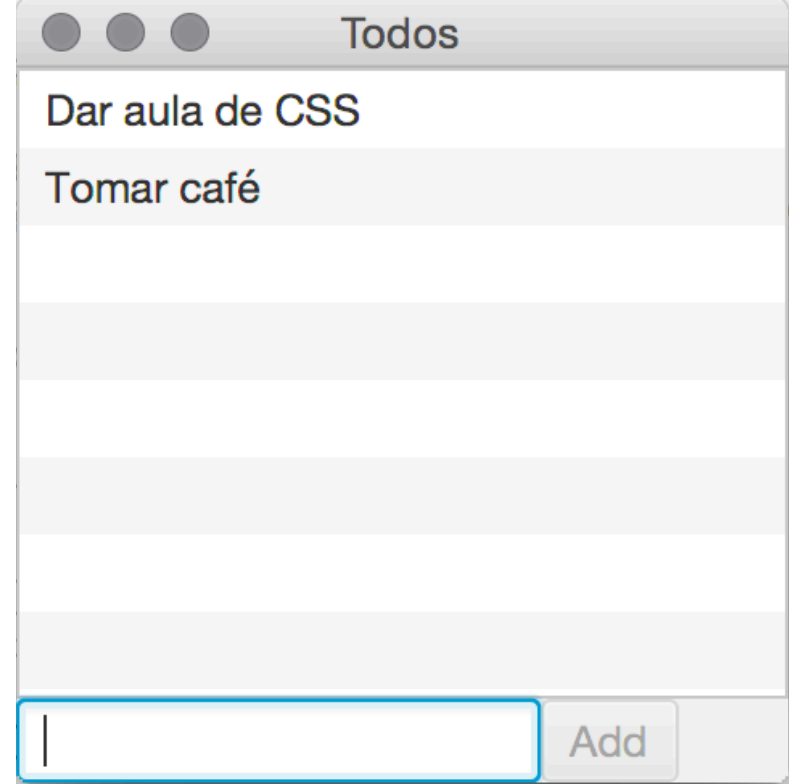
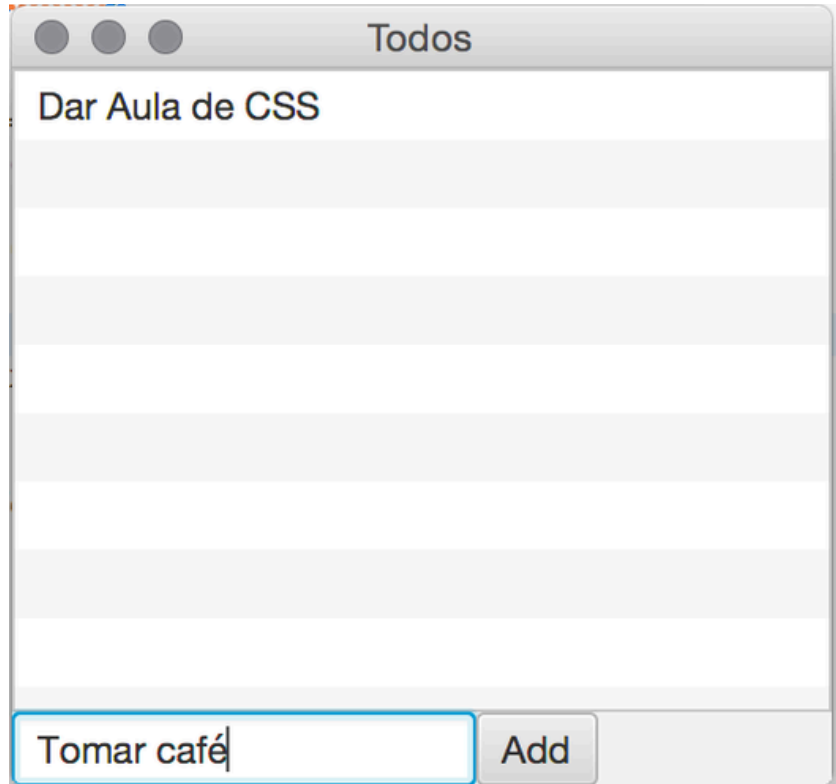
JavaFX

Hierarquia de componentes



JavaFX

Anatomia da aplicação: Exemplo



UI Controls

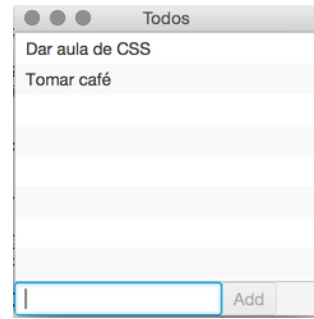
eventos

- Uma grande variedade de *UI Controls*
 - **Button**
 - *TextArea, TextField, ChoiceBox, ComboBox, ListView, MenuBar, Slider, TreeView,...*
- Ações do utilizador sobre o componente propagadas através do lançamento de eventos
 - **ActionEvent**
 - *TreeModificationEvent, InputEvent,...*
- Funcionalidade pretendida definida recorrendo à definição de *Event handlers*

```
void setOnAction(EventHandler<ActionEvent> value){  
    ...  
    Button button = new Button("Click Me");  
    button.setOnAction (e -> btnClicked());  
    ...  
}  
  
private void btnClicked(){  
    System.out.println("Hello World");  
}
```

UI Controls

dados



- Os dados que os *UI Controllers* decidem que é preciso mostrar
 - eg, numa *ListView<String>*
- podem ser obtidos recorrendo a **Observers**
 - eg, populando a *ListView* com uma *ObservableList<String>*
 - mudanças de estado desta lista são observadas pela *ListView*

```
ObservableList<String> todos
```

```
ListView<String> listView = new ListView<>(todos);
```

- estas mudanças podem, por exemplo, ser em reação a uma ação do utilizador sobre *outros UI controllers* como *TextField* e um *Button*

```
addButton.setOnAction(e -> {  
    todos.add(inputField.getText());  
    inputField.setText("");  
    inputField.requestFocus();  
});
```

FXML

- Linguagem baseada em XML que permite **declarativamente** definir a **estrutura** e o **layout** do UI

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.layout.StackPane?>

<StackPane fx:controller="ComponentController" xmlns:fx="http://javafx.com/fxml/1" >
    <children>
        <Button fx:id="button" mnemonicParsing="false" onAction="#sayHello" text="Click Me" />
    </children>
</StackPane>
```

- Em alternativa a fazê-lo **procedimentalmente**

```
Button button = new Button("Click Me");
button.setOnAction (e -> btnClicked("Ola!"));
```

```
StackPane pane = new StackPane();
pane.getChildren().add(button);
```

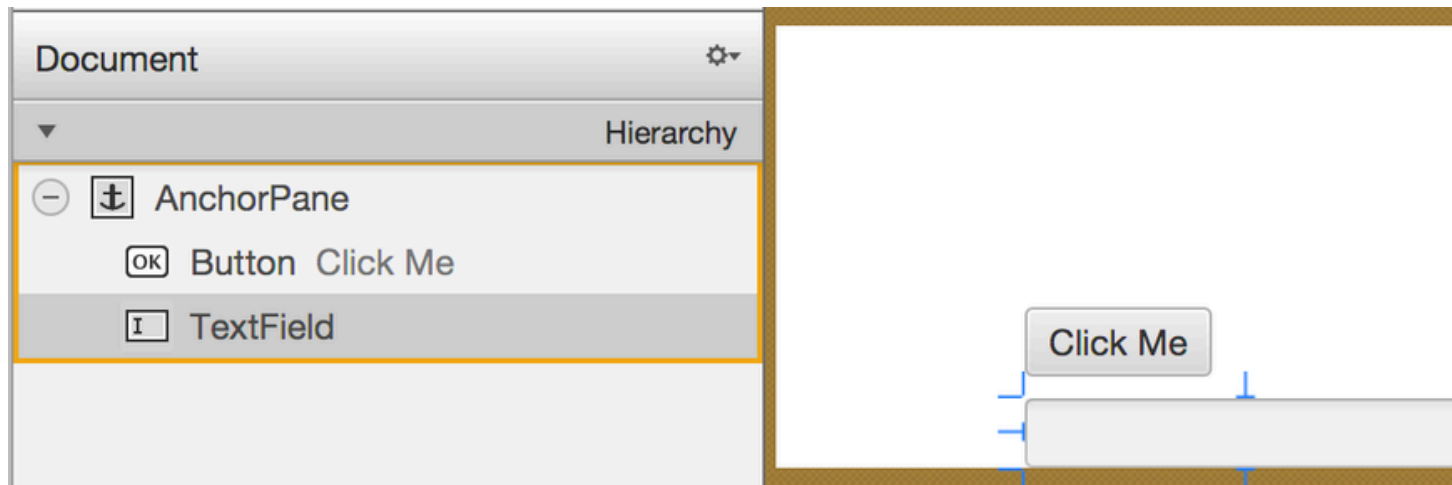
FXML

- A **estrutura** e o **layout** do UI podem ser definidas com a ajuda de ferramentas como o **SceneBuilder**

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Button?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.AnchorPane?>

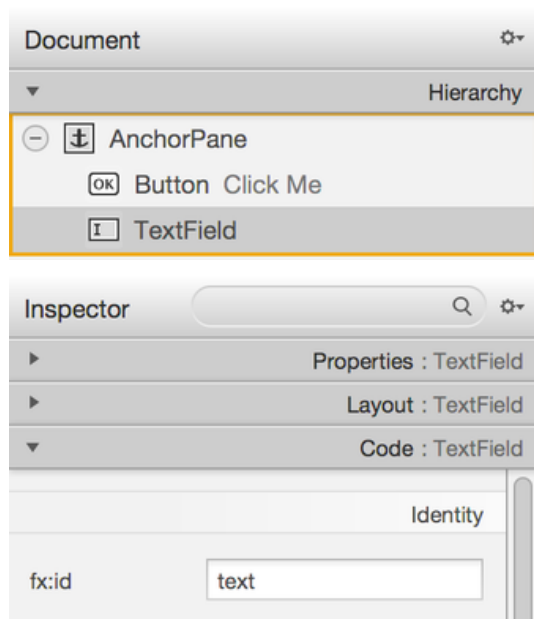
<AnchorPane xmlns:fx="http://javafx.com/fxml/1" xmlns="http://javafx.com/javafx/8.0.111"
    fx:controller="ComponentController">
    <children>
        <Button fx:id="button" layoutX="41.0" layoutY="105.0" mnemonicParsing="false"
            onAction="#sayHello" text="Click Me" />
        <TextField fx:id="text" editable="false" layoutX="-13.0" layoutY="151.0" />
    </children>
</AnchorPane>
```



FXML

Cada componente definido por **FXML File** + **Controller Class**

- o fxml define a **View** do componente e tem uma ligação explícita para a respetiva **Controller Class**
- a *controller class* define o **Controller** e contem
 - o código que é necessário executar quando o utilizador interage com aquele componente do UI
 - atributos onde são automaticamente injetados nós da vista (**injeção de dependências**) marcados com um **fx:id** único no FXML



```
public class ComponentController {  
  
    @FXML  
    private Button button;  
  
    @FXML  
    private TextField text;  
  
    @FXML  
    void sayHello(ActionEvent event) {  
        text.setText(button.getText()+" Button Pressed!");  
    }  
}
```

FXML

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Button?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.AnchorPane?>

<AnchorPane xmlns:fx="http://javafx.com/fxml/1" xmlns="http://javafx.com/javafx/8.0.111"
    fx:controller="ComponentController">
    <children>
        <Button fx:id="button" layoutX="41.0" layoutY="105.0" mnemonicParsing="false"
            onAction="#sayHello" text="Click Me" />
        <TextField fx:id="text" editable="false" layoutX="-13.0" layoutY="151.0" />
    </children>
</AnchorPane>
```

```
public class ComponentController {

    @FXML
    private Button button;

    @FXML
    private TextField text;

    @FXML
    void sayHello(ActionEvent event) {
        text.setText(button.getText()+" Button Pressed!");
    }

}
```

FXML

Startup

- Integração dos vários componentes é definida na **Startup Class**
 - classe que estende **javafx.application.Application**
 - carrega cada ficheiro FXML, o que cria um nó e um controlador
 - define como se organizam os vários nós criados
 - usa o nó raiz para definir uma **scene** e coloca-a no **primary stage**
 - lança a aplicação

```
AnchorPane root = FXMLLoader.load(getClass().getResource("example.fxml"));  
Scene scene = new Scene(root, 300, 275);
```

```
primaryStage.setScene(scene);  
primaryStage.setWidth(400);  
primaryStage.setHeight(300);  
primaryStage.show();
```

Propriedades

Startup

- Um interface definido pelo JavaFx

`javafx.beans.property`

Interface Property<T>

que serve para embrulhar tipos existentes, adicionando-lhe a funcionalidade de notificação de mudança.

- Uma **propriedade**
 - tem métodos **get()** and **set()** para obter e mudar o valor da propriedade (do tipo T)
 - A propriedade é **observável** no sentido em é emitido um evento quando o seu valor muda
- Existem implementações *default* para todos os tipos de propriedades
 - **SimpleIntegerProperty implements Property<Integer>**
 - **SimpleStringProperty implements Property<String>**
 - ...
 - **SimpleObjectProperty<T> implements ObjectProperty<T>**

Data binding

javafx.beans.property

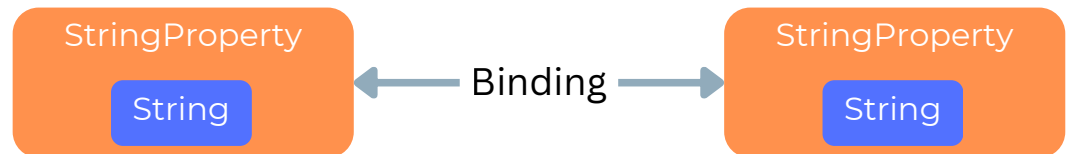
Interface Property<T>

All Superinterfaces:

Observable, ObservableValue<T>, ReadOnlyProperty<T>, WritableValue<T>

- Uma **propriedade**

- é *bindable*, i.e., pode ser ligada a uma outra propriedade do mesmo tipo
- para isso tem métodos **bind(...)**, **bindBidirectional(...)**
- as properties ligadas desta forma ficam forçadas a ter o mesmo valor



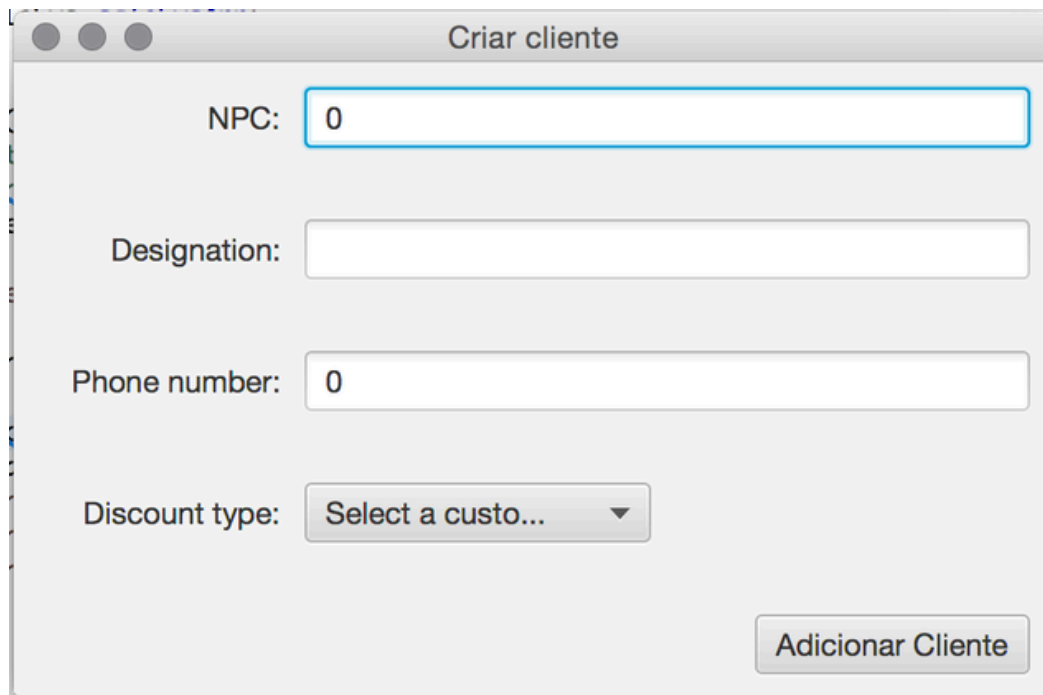
- **Data Binding**

- é o mecanismo que permite definir a sincronização automática dos valores de duas propriedades
- pode ser unidirecional ou bidirecional

```
addButton.disableProperty()  
    .bind(Bindings.isEmpty(inputField.textProperty()));
```

Propriedades

- São um ingrediente importante na definição das classes *ViewModel* usadas para encapsular os dados associados a uma *View*

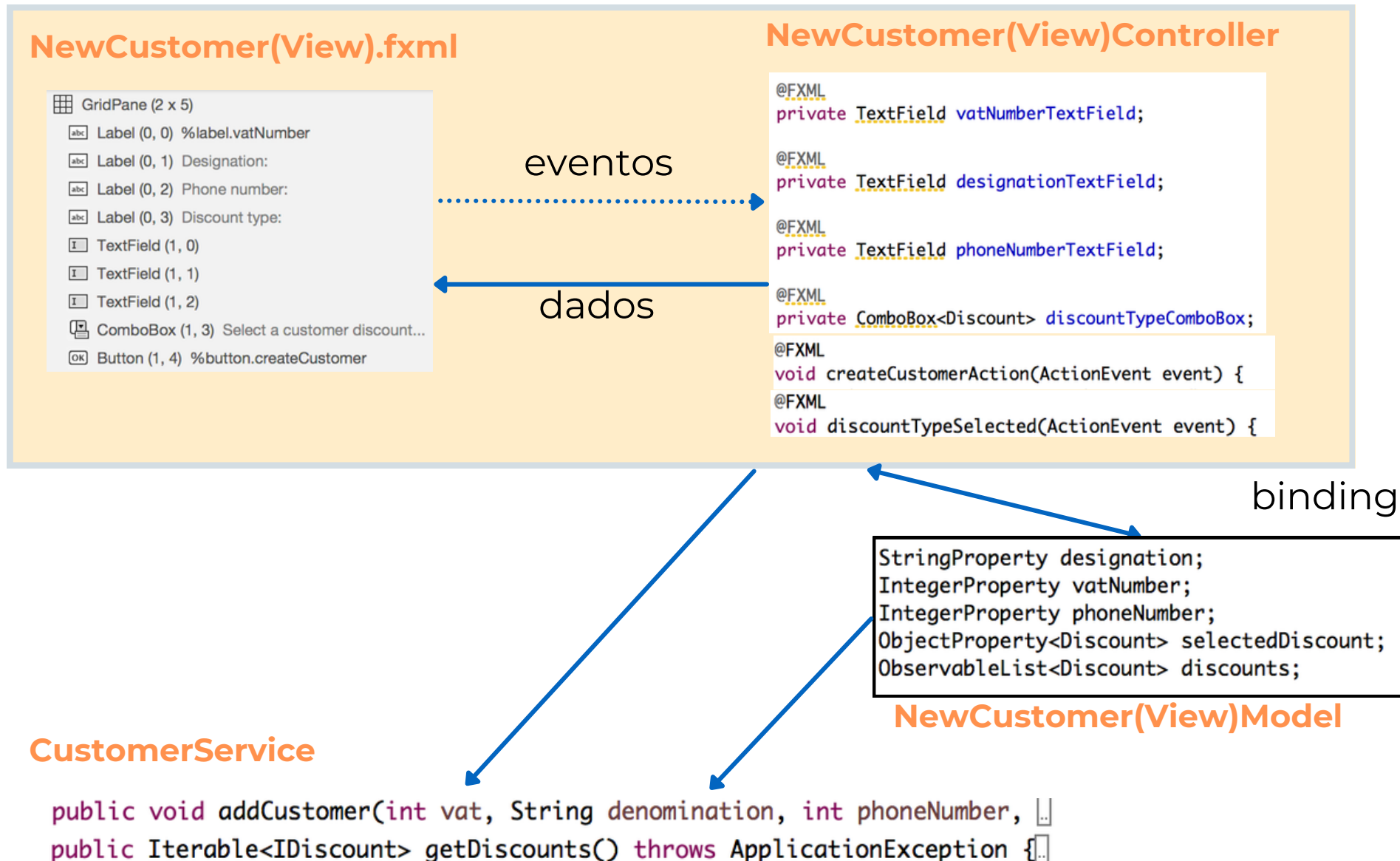


A screenshot of a web application window titled "Criar cliente". The window contains a form with the following fields:

- NPC:** A text input field containing the value "0".
- Designation:** An empty text input field.
- Phone number:** A text input field containing the value "0".
- Discount type:** A dropdown menu with the text "Select a custo..." and a downward arrow.

At the bottom right of the form is a button labeled "Adicionar Cliente".

Propriedades



Propriedades

Inicialização

```
public class NewCustomerModel {  
  
    private final StringProperty designation;  
    private final IntegerProperty vatNumber;  
    private final IntegerProperty phoneNumber;  
    private final ObjectProperty<Discount> selectedDiscount;  
    private final ObservableList<Discount> discounts;  
  
    public NewCustomerModel(CustomerService cs) {  
        designation = new SimpleStringProperty();  
        vatNumber = new SimpleIntegerProperty();  
        phoneNumber = new SimpleIntegerProperty();  
        this.discounts = FXCollections.observableArrayList();  
        try {  
            cs.getDiscounts().forEach(d -> discounts.add(new Discount(d.getDescription(), d.getId())));  
        } catch (ApplicationException e) {  
            // no discounts added  
        }  
        selectedDiscount = new SimpleObjectProperty<>(null);  
    }  
}
```


Propriedades

Data binding

```
public class NewCustomerModel {  
  
    private final StringProperty designation;  
    private final IntegerProperty vatNumber;  
    private final IntegerProperty phoneNumber;  
    private final ObjectProperty<Discount> selectedDiscount;  
    private final ObservableList<Discount> discounts;
```

```
public class NewCustomerController extends BaseController {
```

```
    @FXML  
    private TextField vatNumberTextField;
```

```
    @FXML  
    private TextField designationTextField;
```

```
    @FXML  
    private TextField phoneNumberTextField;
```

```
    @FXML  
    private ComboBox<Discount> discountTypeComboBox;
```

```
public void setModel(NewCustomerModel model) {  
    this.model = model;  
    designationTextField.textProperty().bindBidirectional(model.designationProperty());  
    vatNumberTextField.textProperty().bindBidirectional(model.vatNumberProperty(), new NumberStringConverter());  
    phoneNumberTextField.textProperty().bindBidirectional(model.phoneNumberProperty(), new NumberStringConverter());  
    discountTypeComboBox.setItems(model.getDiscounts());  
    discountTypeComboBox.setValue(model.getSelectedDiscount());  
}
```

Tratamento de eventos + data binding

```
public class NewCustomerModel {  
  
    private final StringProperty designation;  
    private final IntegerProperty vatNumber;  
    private final IntegerProperty phoneNumber;  
    private final ObjectProperty<Discount> selectedDiscount;  
    private final ObservableList<Discount> discounts;  
  
  
    public class NewCustomerController extends BaseController {  
  
        @FXML  
        private TextField vatNumberTextField;  
  
        @FXML  
        private TextField designationTextField;  
  
        @FXML  
        private TextField phoneNumberTextField;  
  
        @FXML  
        private ComboBox<Discount> discountTypeComboBox;  
  
        @FXML  
        void discountTypeSelected(ActionEvent event) {  
            model.setSelectedDiscount(discountTypeComboBox.getValue());  
        }  
    }  
}
```

Tratamento de eventos + data binding

```
public class NewCustomerController extends BaseController {

    @FXML
    private TextField vatNumberTextField;

    @FXML
    private TextField designationTextField;

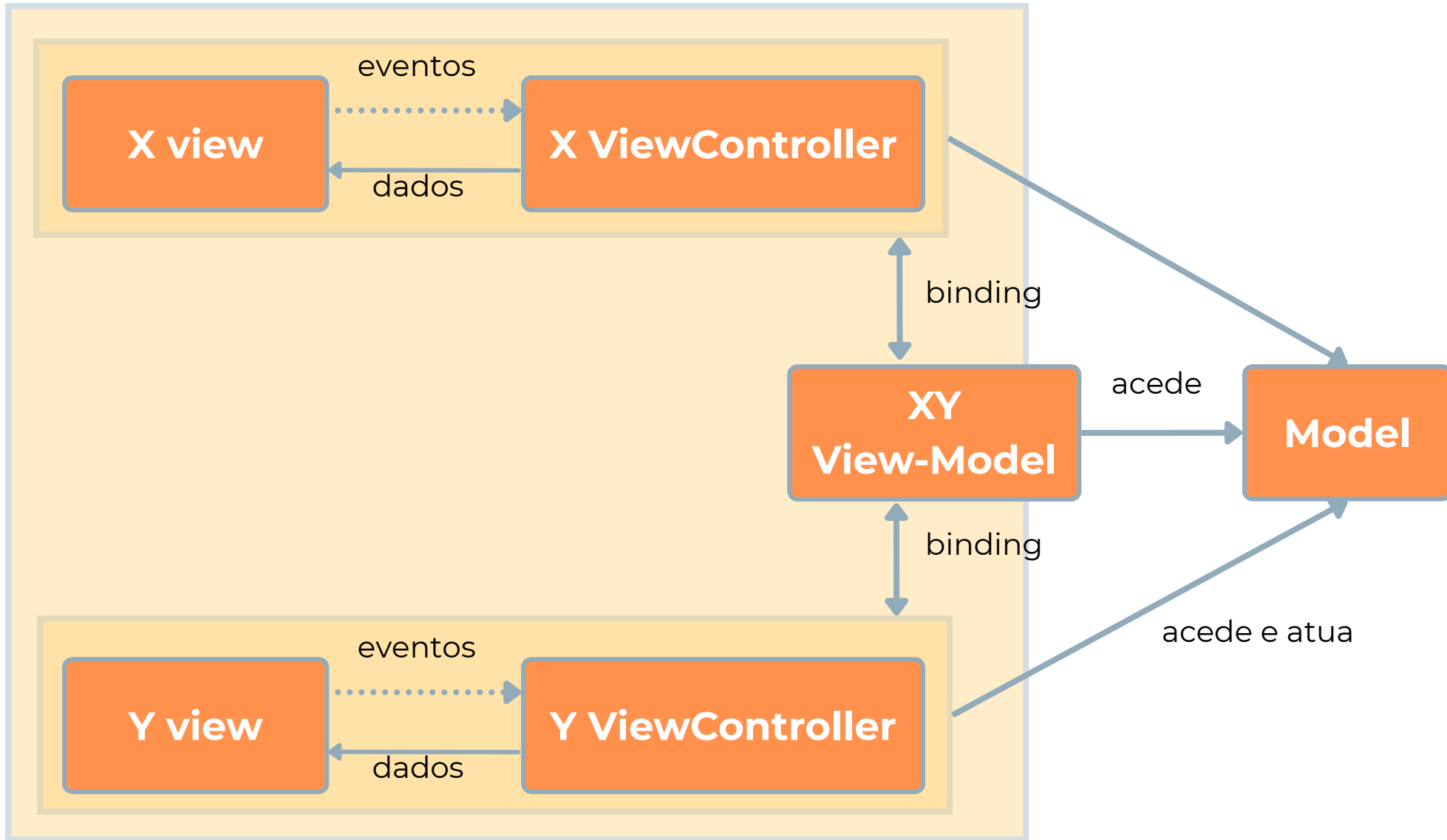
    @FXML
    private TextField phoneNumberTextField;

    @FXML
    private ComboBox<Discount> discountTypeComboBox;

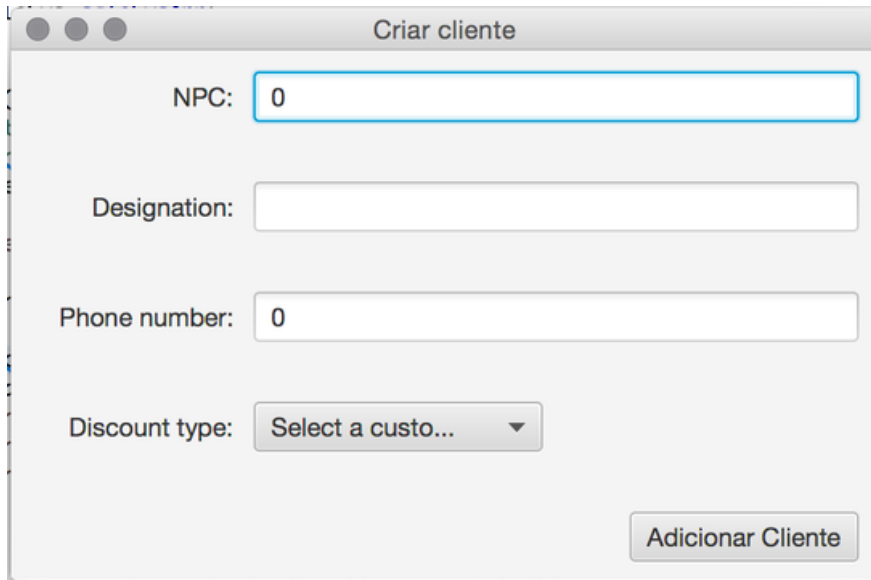
    @FXML
    void createCustomerAction(ActionEvent event) {
        String errorMessages = validateInput();

        if (errorMessages.length() == 0) {
            try {
                customerService.addCustomer(model.getVatNumber(),
                    model.getDesignation(), model.getPhoneNumber(), model.getSelectedDiscount().getDiscountCode());
                model.clearProperties();
                showInfo(i18nBundle.getString("new.customer.success"));
            } catch (VatInvalidException e) {
                showError(i18nBundle.getString("new.customer.error.vatInvalid") + ": " + e.getMessage());
            } catch (ApplicationException e) {
                showError(i18nBundle.getString("new.customer.error.adding") + ": " + e.getMessage());
            }
        } else
            showError(i18nBundle.getString("new.customer.error.validating") + ":\n" + errorMessages);
    }
}
```

JavaFx e os Padrões

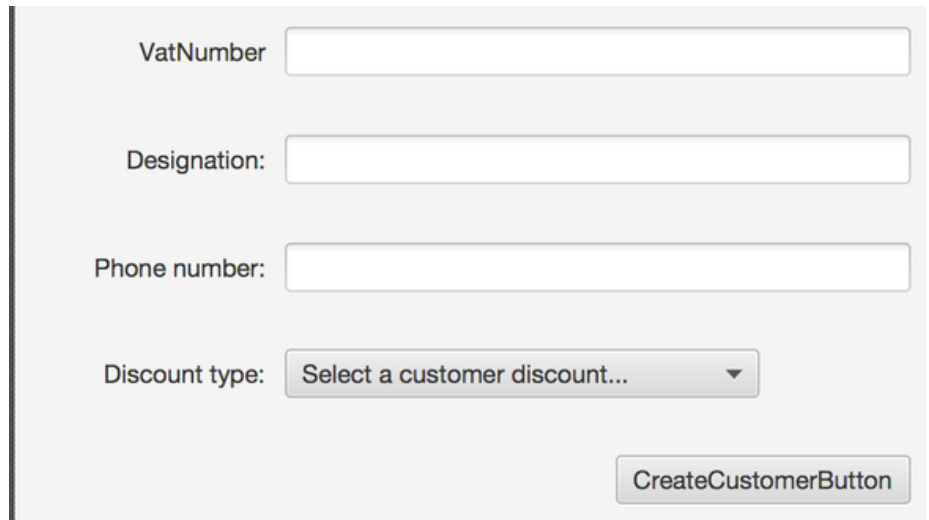


Suporte para a internacionalização



The screenshot shows a Java Swing window titled "Criar cliente". It contains a form with the following elements:

- NPC: A text field containing the value "0".
- Designation: A text field.
- Phone number: A text field containing the value "0".
- Discount type: A dropdown menu with the text "Select a custo..." and a downward arrow.
- Adicionar Cliente: A button at the bottom right.



The screenshot shows a Java Swing window with a form containing the following elements:

- VatNumber: A text field.
- Designation: A text field.
- Phone number: A text field.
- Discount type: A dropdown menu with the text "Select a customer discount..." and a downward arrow.
- CreateCustomerButton: A button at the bottom right.

```
showInfo(i18nBundle.getString("new.customer.success"));
```

```
ResourceBundle i18nBundle = ResourceBundle.getBundle("i18n.Bundle", new Locale("pt", "PT"));
```

```
FXMLLoader createCustomerLoader = new FXMLLoader(getClass().getResource("/fxml/newCustomer.fxml"), i18nBundle);
```

```
<Label text="%label.vatNumber" GridPane.halignment="RIGHT" />
```

Bundle_pt_PT.properties

```
1 application.title = Criar cliente
2 label.vatNumber = NPC:
3 button.createCustomer = Adicionar Cliente
4
5 error.dialog.title = Erro
6 info.dialog.title = Sucesso
7
```

Suporte para a internacionalização



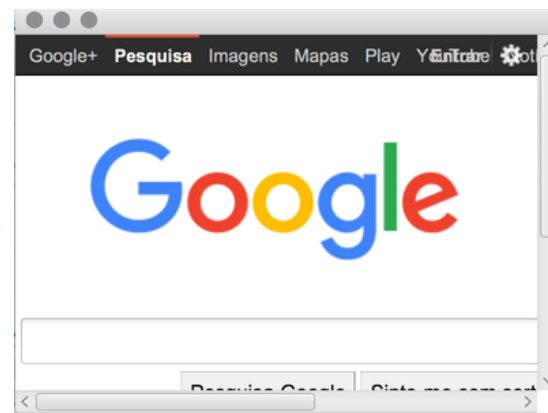
Bundle_pt_PT.properties ✕

```
1 application.title = Criar cliente
2 label.vatNumber = NPC:
3 button.createCustomer = Adicionar Cliente
4
5 error.dialog.title = Erro
6 info.dialog.title = Sucesso
7
8 new.customer.error.adding = Erro ao criar o cliente
9 new.customer.error.validating = Erro ao validar cliente
10 new.customer.error.vatInvalid = Número de pessoa coletiva inválido
11 new.customer.success = Cliente criado com sucesso
12 new.customer.invalid.designation = A designação tem de ser preenchida
13 new.customer.invalid.VATNumber = O número de pessoa coletiva tem de
14 new.customer.invalid.discount = O tipo de desconto tem de ser preenchido
```


WebView e CSS

- O componente **WebView** permite embeber conteúdo web na aplicação, nomeadamente mostrar páginas definidas por exemplo com HTML5 e CSS

```
StackPane pane = new StackPane();  
WebView webView = new WebView();  
WebEngine engine = webView.getEngine();  
//Load a web page  
engine.load("http://www.google.com");  
//Add the web view to the JavaFX view  
pane.getChildren().add(webView);
```



- É baseado no **WebKit** (*web pages render engine*) e torna possível capitalizar numa aplicação web para fazer uma aplicação desktop, juntando-lhe funcionalidades que só faz sentido ter neste contexto, como por exemplo o acesso ao disco
- Pode se definir o estilo **CSS** a aplicar a um **scene graph** ou a um nó específico

UI no Android



Sistema Operativo + ferramentas + APIs

Ao desenvolver interfaces gráficas, alguns fatores devem ser levados em conta:

- Otimizadas para interações com os dedos
- Interrupções (chamadas, notificações) e mudanças de configuração (e.g., rotação da tela).
- O layout deve ser adaptável a milhares de modelos de dispositivos.

UI no Android



Sistema Operativo + ferramentas + APIs

Ao desenvolver interfaces gráficas, alguns fatores devem ser levados em conta:

- Otimizadas para interações com os dedos
- Interrupções (chamadas, notificações) e mudanças de configuração (e.g., rotação da tela).
- O layout deve ser adaptável a milhares de modelos de dispositivos.

Há duas abordagens para UI de aplicações Android:

- **Views e XML:** Abordagem clássica com (modelo imperativo)
- **Jetpack Compose:** Abordagem recomendada desde 2021 com (modelo declarativo)

Views e XML

- Estrutura estática da UI é definida com ficheiros XML. O código (Kotlin/Java) altera o estado da UI de acordo com os eventos.
- **XML**: Descreve a hierarquia de componentes (Views) e seus atributos (tamanho, cor, posição).
- **Activity/Fragment**: Construído por cima do XML, obtém referências aos componentes (ex: *findViewById*) e adiciona lógica a eles (ex: *button.setOnClickListener*).

Exemplo: views e XML

login_activity.xml

```
<androidx.constraintlayout.widget.ConstraintLayout ...>

    <ImageView
        android:id="@+id/logo"
        ... />

    <EditText
        android:id="@+id/input_email"
        android:hint="Email"
        app:layout_constraintTop_toBottomOf="@id/logo"
        ... />

    <Button
        android:id="@+id/login_button"
        android:text="Entrar"
        app:layout_constraintTop_toBottomOf="@id/input_email"
        ... />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Exemplo: views e XML

MainActivity.kt

ou

FirstFragment.kt

```
// No Activity ou Fragment
```

```
val loginButton = findViewById<Button>(R.id.login_button)
```

```
loginButton.setOnClickListener {
```

```
    /* Lógica do login */
```

```
}
```

```
<androidx.constraintlayout.widget.ConstraintLayout ...>

    <ImageView
        android:id="@+id/logo" login_activity.xml
        ... />

    <EditText
        android:id="@+id/input_email"
        android:hint="Email"
        app:layout_constraintTop_toBottomOf="@id/logo"
        ... />

    <Button
        android:id="@+id/login_button"
        android:text="Entrar"
        app:layout_constraintTop_toBottomOf="@id/input_email"
        ... />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Exemplo: views e XML

login_activity.xml

```
<androidx.constraintlayout.widget.ConstraintLayout ...>

    <ImageView
        android:id="@+id/logo"
        ... />

    <EditText
        android:id="@+id/input_email"
        android:hint="Email"
        app:layout_constraintTop_toBottomOf="@id/logo"
        ... />

    <Button
        android:id="@+id/login_button"
        android:text="Entrar"
        app:layout_constraintTop_toBottomOf="@id/input_email"
        ... />

</androidx.constraintlayout.widget.ConstraintLayout>
```

MainActivity.kt

```
// No Activity ou Fragment
val loginButton = findViewById<Button>(R.id.login_button)
loginButton.setOnClickListener {
    /* Lógica do login */
}
```

Views e XML

- **Paralelo com JavaFX:**

- O *XML* é análogo ao *FXML*.
- A **Activity** ou **Fragment** age de forma similar ao **Controller**
- Classes como **ConstraintLayout** são similares ao `AnchorPane` ou `GridPane` (com muito mais recursos)

JetPack Compose

- A UI é descrita para ter um estado. A framework atualiza o ecrã quando o estado muda.
- A UI é descrita diretamente em Kotlin através do uso de *Composables*.
- Diminui a separação entre layout e lógica → código mais coeso e legível
- Mais alinhado com frameworks modernos como React ou SwiftUI.

Exemplo: JetPack Compose

```
@Composable
fun LoginScreen() {
    // O estado para controlar o valor do email
    var email by remember { mutableStateOf("") }

    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Image(...) // Logo

        OutlinedTextField(
            value = email,
            onChange = { email = it }, // atualiza o estado
            label = { Text("Email") }
        )

        Button(onClick = { /* Lógica do login */ }) {
            Text("Entrar")
        }
    }
}
```


Camada de Apresentação

Rich Client Applications

Camada de Apresentação

Rich Client Applications